

Progmune Runtime 内部技术白皮书

v2.2

程序免疫学：约束引导的程序合成运行时 —— 逻辑框架与技术细节

★ v2.2 核心跨越: Semantic Ledger Runtime — 事件溯源架构，纯函数不变式系统，可重放、可查询、可证明的账本运行时

Progmune 团队

2026-05-30

第一章 概述与设计哲学

1.1 问题背景

大语言模型（LLM）在代码生成领域取得了显著进展，但 LLM 输出的代码存在固有的不确定性问题：幻觉函数名、类型不匹配、变量未定义即使用、违反 API 调用协议等。传统方法依赖人工审查或单元测试来捕获这些错误，但人工审查不可扩展，单元测试覆盖有限。

Progmune Runtime 提出了一种根本不同的方案：**程序免疫学（Program Immunology）**——借鉴生物免疫系统识别和清除病原体的原理，构建多层约束验证运行时，自动检测、诊断并修复 LLM 生成代码中的语义异常。

1.2 核心设计原则

原则一：约束即规范（Constraints as Specifications）

程序的合法空间由约束定义，而非由生成模型定义。任何 LLM 输出必须通过四层语义验证层（SVL-1 到 SVL-4）才能被接受。

原则二：免疫记忆（Immune Memory）

每一次约束违规都是一次"感染事件"。系统记录违规的模式、上下文和修复路径，形成"抗体"。当相同或相似的意图再次出现时，免疫记忆可以跳过 LLM 推理直接提供已验证的解，或注入修复提示。

原则三：确定性修复（Deterministic Repair）

当协议违规有已知修复路径时，系统不应回退到 LLM 重新生成，而应通过 SSG 状态图的 BFS 搜索自动插入缺失的函数调用。

原则四：可观测性优先（Observability First）

每次规划会话产生完整的执行记录：Attempt（每次尝试）、StateTransition（状态转移）、ConstraintViolation（约束违规）。这些记录支持终端回放、Web 仪表盘和统计分析。

| ★ v2.2 原则五：事件溯源（Event Sourcing）

v2.1 的 StateTransition[] 是验证过程的被动日志——真正的真相源是 StateMachineValidator 实例中的可变状态。v2.2 反转这一关系：

StateTransition[] 成为系统的唯一真相源（Single Source of Truth），命名空间状态是从 Ledger 中派生出来的视图，而非独立存储的可变对象。这意味着：任何时刻的系统状态都可以通过从空状态出发重放完整的 StateTransition 序列来重建；任何验证决策都可以被独立审计和复现；约束规则的变更可以被检测和追溯。

1.3 系统定位

Progmune Runtime 是一个 **MCP (Model Context Protocol) 服务器**，作为 LLM 与目标代码库之间的约束验证中间层。它不替代 LLM，而是为 LLM 提供一个"免疫沙箱"——LLM 输出在沙箱中经过多层验证后才能进入代码库。

1.4 v2.1 → v2.2 版本演进 | ★ v2.2

v2.1 完成了运行时本体论 (Runtime Ontology) 的建立——将 Attempt、StateTransition、ConstraintViolation、ExecutionSession 提升为第一类类型，并构建了以 StateMachineValidator 为核心的有状态验证架构。但在这个架构中，StateTransition[] 只是被动日志——决策来自 StateMachineValidator 实例中的可变状态，而非来自不可变的事件记录。

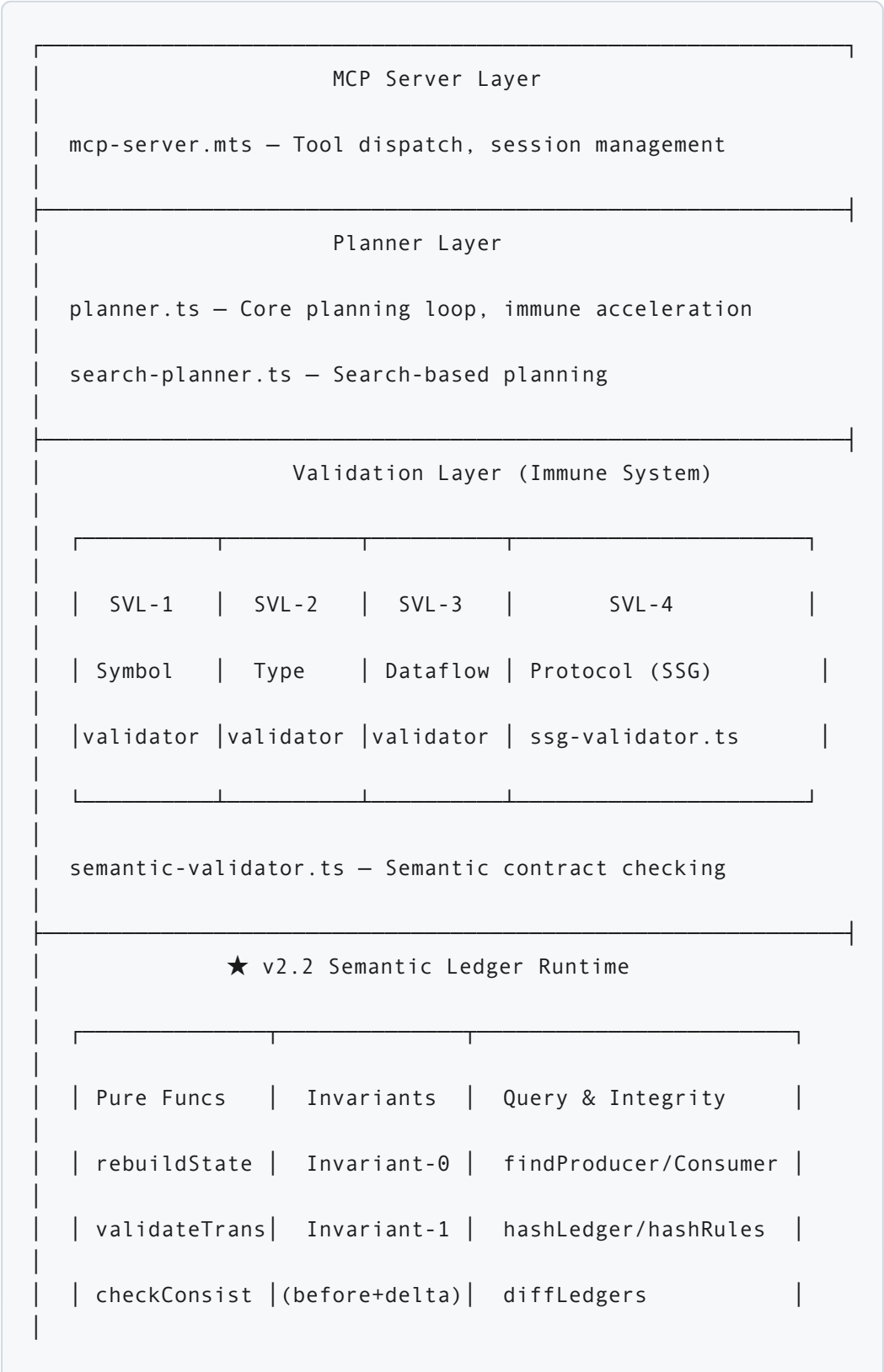
v2.2 的核心跨越是将架构从 **Event Logging (事件记录)** 推进到 **Event Sourcing (事件溯源)**。具体而言：

维度	v2.1 (Event Logging)	v2.2 (Event Sourcing)
真相源	StateMachineValidator 可变实例	StateTransition[] 不可变账本
状态获取	ssv.snapshotNamespaceStates() 读可变状态	rebuildState(ledger) 纯函数派生
验证逻辑	StateMachineValidator.apply() 有状态方法	validateTransition() 纯函数
	无系统化不变式	

维度	v2.1 (Event Logging)	v2.2 (Event Sourcing)
一致性保证		Invariant-0 (历史一致性) + Invariant-1 (增量自洽)
约束追溯	无	ruleHash (SHA256 约束快照) 贯穿所有层级
账本完整性	无	hashLedger() (SHA256 防篡改指纹)
查询能力	遍历日志文件	5 个纯查询函数 (findProducer/Consumer 等)
跨会话比较	无	diffLedgers() 结构化差异
向后兼容	—	StateMachineValidator 类保留，内部委托给纯函数

第二章 核心架构

2.1 系统分层



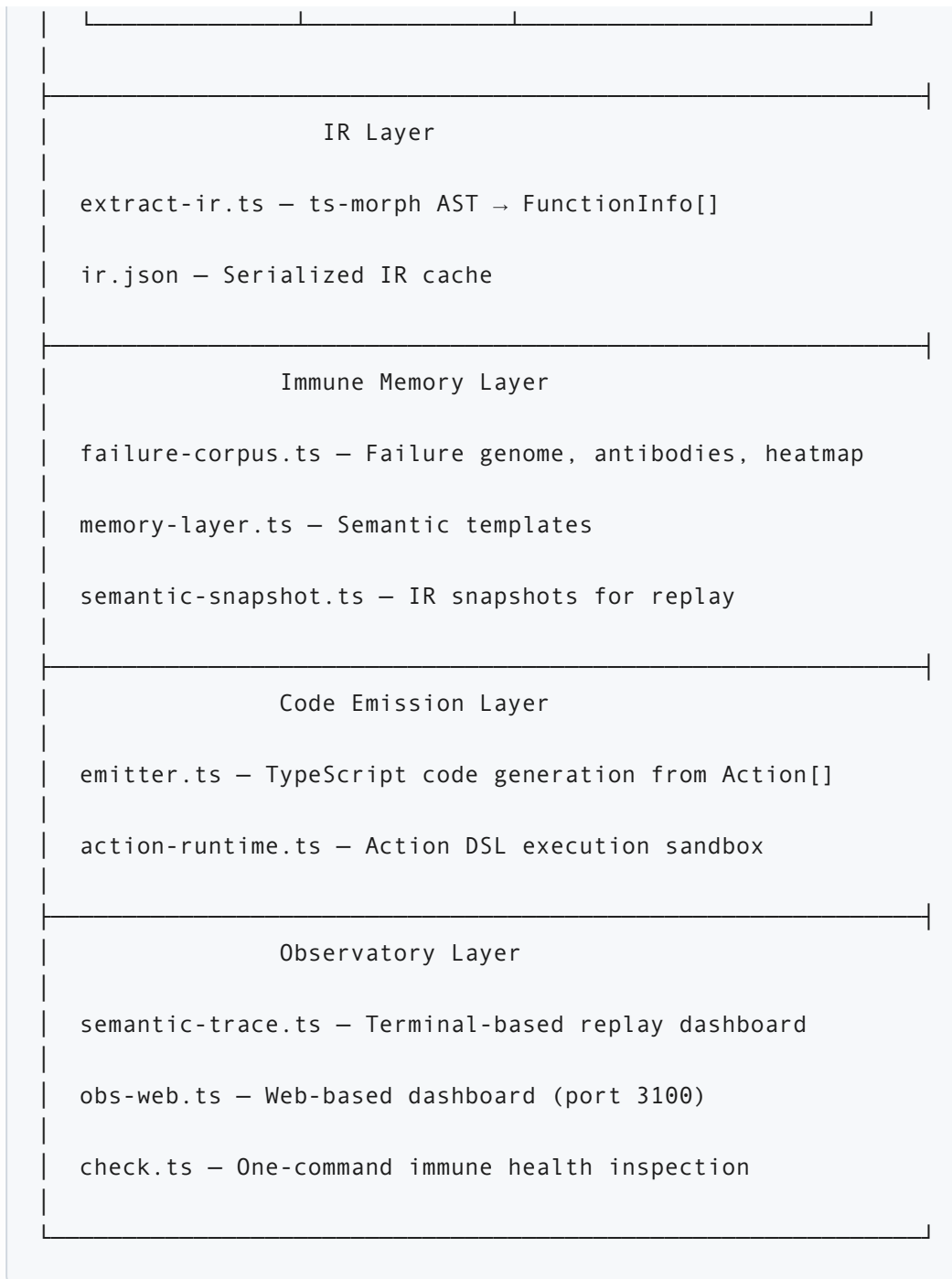
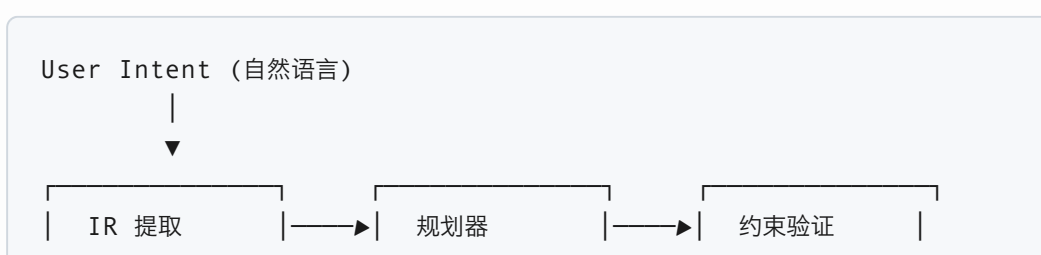


图 2.1: 系统分层架构 (★ v2.2 新增 Semantic Ledger Runtime 层, 位于验证层与 IR 层之间, 是整个系统的状态派生引擎)

2.2 核心数据流



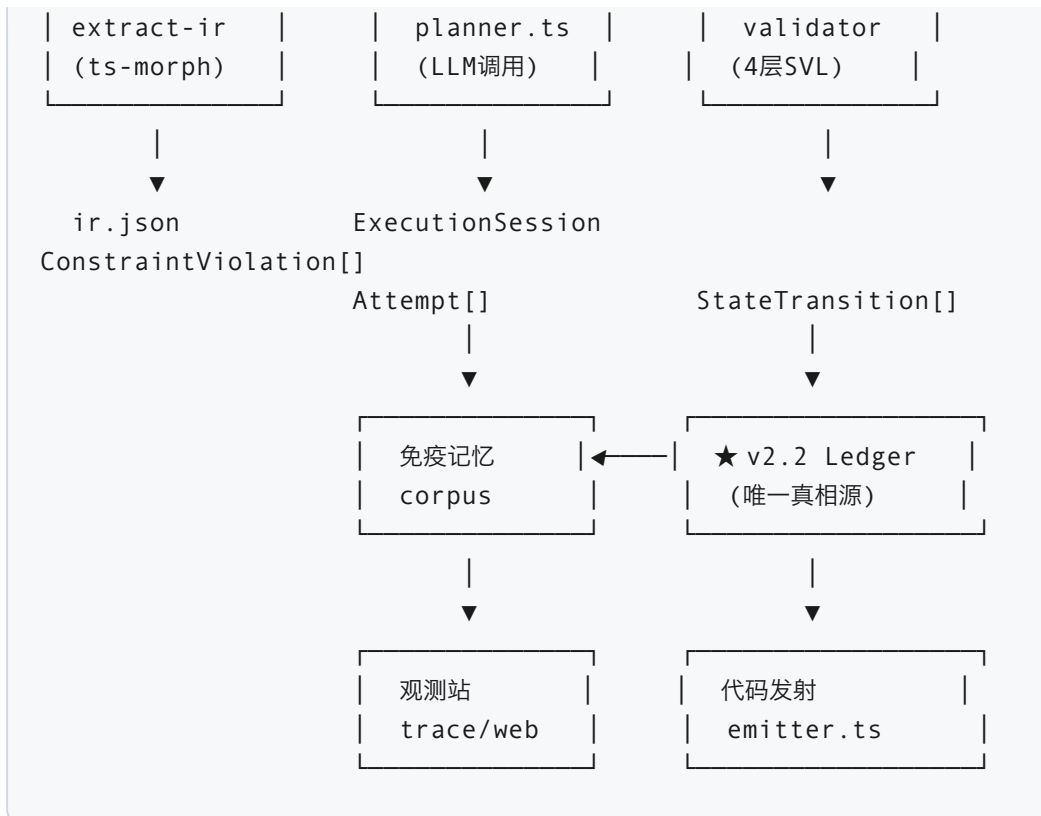


图 2.2: 核心数据流 (★ v2.2: Transition Ledger 取代 StateMachineValidator 成为系统中心, 所有下游消费者从 Ledger 派生状态)

2.3 ★ v2.2 架构跨越: Event Logging → Event Sourcing

这是 v2.2 最根本的架构变更。理解这一跨越, 是理解整个 Semantic Ledger Runtime 的前提。

2.3.1 v2.1 模式: Event Logging

```

StateMachineValidator (可变状态, 真相源)
|
| apply(functionName, actionIndex)
| → 更新内部 this.state (Map<ns, Set<state>>)
| → 返回 { valid, transition }
|
▼
StateTransition[] (被动日志, 非真相源)
  
```

问题:

- 真相在可变对象中, 无法从日志独立重建
- 验证逻辑与状态耦合, 无法独立测试

- 无法审计"为什么这个转移被接受"
- 每条转移的验证依赖前一条转移的副作用

2.3.2 v2.2 模式: Event Sourcing

```
StateTransition[] (不可变账本, 唯一真相源)
|
| rebuildState(ledger) → 派生状态视图
| validateTransition(ctx, fn, i, rules, nsInit,
ruleHash) → 纯函数验证
▼
CurrentState (派生视图, 非真相源)
|
|— Replayable: 任何人都可以从空状态重放 Ledger 得到相同结果
|— Queryable: 5 个纯查询函数在 Ledger 上运行
|— Provable: hashLedger + ruleHash 提供防篡改完整性证明
```

优势:

- 真相在不可变账本中, 任何时刻的状态都是确定性的派生结果
- 验证逻辑是纯函数, 可在任何上下文中独立测试
- 每条转移携带完整的前后状态快照, 支持独立审计
- 约束变更可通过 ruleHash 检测和追溯

2.4 Strangler Pattern 向后兼容策略

StateMachineValidator 被 planner.ts、semantic-trace.ts、check.ts、p0_ssg_demo.ts 等多个文件使用。v2.2 不删除这个类, 而是将其重构为向后兼容的包装器:

```
StateMachineValidator (保留, 向后兼容包装器)
|
|— apply() → 委托给 validateTransition()
|— applyWithTransition() → 委托给 validateTransition()
|— snapshotNamespaceStates() → 委托给
rebuildState(this.ledger)
|— getTrace() → 返回 this.ledger
```

新代码直接使用纯函数:

```
planner.ts → validateTransition() + ValidationContext
semantic-trace.ts → rebuildState() +
checkLedgerConsistency()
```



```
check.ts      → 纯函数循环 + Invariant-0 + Invariant-1
p0_ssg_demo.ts → createPureContext() 零实例化
```

第三章 运行时本体论 (Runtime Ontology)

运行时本体论定义了 Progmune 执行语义的所有基本对象。这些类型集中定义在 `src/runtime-types.ts` 中，是整个系统的单一权威来源 (Single Source of Truth for Types)。

3.1 Action DSL

Action 是 Progmune 中程序行为的原子单位——一个 discriminated union，表示 LLM 生成的每个操作。

```
type Action =
  | { kind: "call"; function: string; args: Arg[]; assignTo?: string }
  | { kind: "assign"; target: string; value: string | Action }
  | { kind: "return"; value: string | Action }
  | { kind: "if"; condition: string; thenActions: Action[];
      elseActions?: Action[] }
  | { kind: "for"; variable: string; iterable: string;
      bodyActions: Action[] }

type Arg = { name: string; type: string; value: string | Action }
```

Action DSL 的设计要点：

1. `call` — 函数调用，`args` 携带静态类型信息 (`name`, `type`, `value`)，`assignTo` 表示返回值绑定
2. `assign` — 变量赋值，`value` 可以是字面量或嵌套 Action
3. `return` — 返回语句，`value` 可以是变量名引用 (以 `$` 前缀区分的 JSON wire format) 或嵌套 Action
4. `if` — 条件分支，`thenActions/elseActions` 各自是完整的 `Action[]`

5. `for` — 循环, `bodyActions` 是循环体

JSON Wire Format

LLM 输出的紧凑 JSON 格式 (用于减少 token 消耗) :

```
[
  {
    "f": "extractIR",
    "to": "ir",
    "a": [
      {
        "n": "root",
        "t": "str",
        "v": "."
      }
    ]
  },
  {
    "f": "validateAction",
    "to": "ok",
    "a": [
      {
        "n": "action",
        "t": "any",
        "v": "$ir"
      }
    ]
  },
  {
    "r": "ok"
  }
]
```

- `f` → function name, `to` → assignTo, `a` → args
- `$变量名` 表示对之前返回值的引用
- `r` → return

解析函数 `parseActionJSON()` 负责将紧凑 JSON 转换为 `Action[]` 。

3.2 ConstraintViolation

每次约束违规都产生一个结构化的 `ConstraintViolation` 对象:

```
interface ConstraintViolation {
  svl: 1 | 2 | 3 | 4; // 违规层级
  violatedConstraint: string; // 违规约束类型
  actionIndex: number; // 违规动作索引
  currentStates?: string[]; // 当前 SSG 状态 (SVL-4)
  requiredStates?: string[]; // 所需 SSG 状态 (SVL-4)
  missingStates?: string[]; // 缺失的前置函数 (SVL-4)
  conflictingStates?: string[]; // 冲突状态
  fixPath?: string[]; // BFS 搜索的修复路径
  namespace?: string; // 命名空间
  description: string; // 人类可读描述
}
```

3.3 StateTransition | ★ v2.2 ruleHash

协议状态机的每次合法调用产生一个 `StateTransition` :

```
interface StateTransition {
  actionIndex: number;
  function: string;
  namespace: string;
  acquired: string[]; // 新获得的状态
  invalidated: string[]; // 被失效的状态
  statesBefore: Record<string, string[]>; // per-
    namespace 快照 (执行前)
  statesAfter: Record<string, string[]>; // per-
    namespace 快照 (执行后)
  valid: boolean;
  ruleHash?: string; // ★ v2.2 约束
    快照
}
```

关键设计决策： `statesBefore` 和 `statesAfter` 使用 `Record<string, string[]>` (per-namespace)，而非合并的 `string[]`。这样每个命名空间 (auth、file、db、dev_pipeline) 的状态变化可以独立追踪。

★ v2.2 ruleHash 字段： 存储创建此转移时协议规则集的 SHA256 哈希。这个字段是实现跨版本确定性回放的关键基础设施——如果 2027 年的规则集与 2026 年不同，回放 2026 年的 Ledger 时会检测到 `ruleHash` 不匹配，系统可以明确报告“确定性回放不可行：约束集已变更”，而不是静默产生不同的结果。同样添加到 `Attempt` 和 `ExecutionSession` 中，形成从单条转移到会话级别的完整约束谱系。

3.4 Attempt | ★ v2.2 ruleHash

一次“尝试”是规划器的单次推理循环：

```
interface Attempt {
  id: string; //
    att_<timestamp>_<random>
  sessionId: string; // 所属会话
  attemptNumber: number; // 1-based
  parentAttemptId?: string;
  inputIntent: string; // 用户意图原文
  plannerSeed: string; // MD5(prompt|model|
    timestamp)
  constraintSnapshotId: string; // IR 快照链接
  generatedActions: Action[]; // LLM 输出
  transitions: StateTransition[]; // SSG 状态转移
  violations: ConstraintViolation[]; // 约束违规列表
}
```

```

outcome: "success" | "constraint_violation" |
        "planner_failure";
timestamp: number;
llmCallCount: number;
durationMs: number;
antibodyHit?: AntibodyHit;           // 如果被免疫加速
ruleHash?: string;                   // ★ v2.2 约束快照
}

```

3.5 ExecutionSession | ★ v2.2 ruleHash

一次完整的规划会话：

```

interface ExecutionSession {
  sessionId: string;
  intent: string;
  attempts: Attempt[];           // 所有尝试（按时间顺序）
  successfulAttempt?: Attempt;   // 成功的尝试
  resolved: boolean;             // 是否最终解决
  snapshotId?: string;          // IR 快照 ID
  ruleHash?: string;             // ★ v2.2 约束快照
  startedAt: number;
  endedAt?: number;
}

```

| ★ v2.2 ruleHash 出现在 StateTransition、Attempt、ExecutionSession 三个层级，形成完整的约束谱系链：

ExecutionSession.ruleHash = Attempt.ruleHash = StateTransition.ruleHash。这个不变式由 planner.ts 在会话初始化时通过 hashRules() 计算一次，然后注入到所有子结构中。

3.6 AntibodyHit

抗体命中记录，捕获免疫系统加速规划器的每次事件：

```

interface AntibodyHit {
  level: string;                 // ACL-1 | ACL-2 | ACL-3 | ACL-4
  signature: string;             // 失效模式签名
  fixPath: string[];             // 修复路径
  similarityScore: number;       // Jaccard 相似度
  action: "fast_path" | "injected_hint";
}

```

```
    llmCallsSaved: number;  
    estimatedTokensSaved: number;  
}
```

3.7 ValidationContext | ★ v2.2 新增

ValidationContext 是 v2.2 纯函数验证循环的核心数据结构，封装了当前账本和派生状态：

```
interface ValidationContext {  
    ledger: StateTransition[];           // 已确认的转  
    // 移列表  
    currentState: Record<string, string[]>; // 派生状态快照  
}
```

使用模式：初始时

`ctx = { ledger: [], currentState: rebuildState([], nsInit) }`。每次验证后，调用方将 `transition` 推入 `ctx.ledger` 并更新 `ctx.currentState`。这使得验证循环从 $O(n^2)$ 降为 $O(n)$ ——不再需要每次在循环内重建整个账本的状态。

第四章 IR 提取系统

4.1 设计目标

IR (Intermediate Representation, 中间表示) 是 Progmune 对目标代码库的模型。它将 TypeScript 源码抽象为结构化的函数签名列表，供 LLM 和验证器使用。IR 是系统与代码库之间的唯一接口——所有约束验证、协议检查、类型匹配都基于 IR 提供的信息。

4.2 技术实现

`extractIR(projectRoot: string): FunctionInfo[]` 使用 `ts-morph` (TypeScript 编译器 API 的包装) 提取函数签名信息。`ts-morph` 提供了比正则表达式更可靠的 AST 级别解析能力。

提取范围

1. **函数声明** (`function foo() {}`) —— 名称、参数、返回类型
2. **箭头函数变量** (`const foo = () => {}`) —— 包括简单箭头函数和由高阶函数 (如 `debounce(() => {})`) 包装的箭头函数。系统会递归检查 `CallExpression` 的参数来发现包装的箭头函数
3. **JSDoc @protocol 注解** —— 解析 `@protocol namespace=xxx pre_states=["A"] post_states=["B"] invalidate=["C"]` 格式, 提取协议状态机定义
4. **直接调用图** —— 每个函数体内调用的其他函数名, 用于 SVL-3 数据流分析和依赖关系推断
5. **外部依赖** —— 从调用图中识别非项目声明但被引用的函数 (如 `fs.readFileSync` 、 `path.resolve`) , 从已知外部函数注册表 (约 40+ 条目) 补充签名信息

类型系统

- 保留原始 TypeScript 类型字符串 (`string` 、 `number` 、 `Promise<User>` 等)
- 提供结构化类型细节 (`typeDetail`) : 联合类型展开 (`string | null`) 、 泛型展开 (`Array<string>`)
- 标准化类型简写: `str` 、 `int` 、 `bool` 、 `dict` 、 `list` 用于 LLM prompt 紧凑表示
- 类型别名展开: 如果参数类型是项目中定义的类型别名, IR 会解析到最终的具体类型

外部函数注册表

系统维护了一个包含约 40 个常用 Node.js/JS 标准库函数的签名注册表，覆盖：

- `fs` 模块 (`readFileSync`、`writeFileSync`、`existsSync`、`mkdirSync`、`readdirSync`、`unlinkSync` 等)
- `path` 模块 (`resolve`、`relative`、`dirname`、`basename`、`extname`、`join` 等)
- `JSON` (`parse`、`stringify`)
- `console` (`log`、`error`、`warn`)
- `process` (`exit`、`cwd`、`env`)
- `crypto` (`createHash`、`digest`)
- `child_process` (`execSync`、`spawn`)
- `setTimeout`、`Math` 等

过滤掉的 JS 内置方法（约 60+ 个）：`map`、`filter`、`reduce`、`toString`、`push`、`forEach`、`find`、`sort`、`slice` 等。这些方法不需要在 IR 中出现，因为它们不参与协议状态机的构建。

协议注解提取

```
function parseProtocolFromJSDoc(node: any):  
    FunctionInfo['protocol'] | undefined {  
    // 解析格式: @protocol namespace=file pre_states=["A","B"]  
    //           post_states=["C"] invalidate=["A"]  
    // 提取: namespace (string), pre_states (string[]),  
    //       post_states (string[]), invalidate (string[]?)  
}
```

JSDoc 注解是协议规则的代码内定义方式，与 `protocols.json` 互补。两种来源合并时，IR 中的 `@protocol` 注解优先，但 JSON 中的 `namespace` 定义作为权威命名空间声明。

4.3 输出格式

`ir.json` 是一个 `FunctionInfo[]` 数组，每个条目包含：

字段	类型	说明
name	string	函数名
params	ParamInfo[]	参数列表（name、type、typeDetail）
returnType	string	返回类型
file	string	源文件路径
calls	string[]	直接调用的函数名
external	boolean?	是否为外部函数
description	string?	外部函数的描述
protocol	object?	JSDoc @protocol 注解

4.4 协议状态注解

IR 提取本身也是 SSG 协议的一部分：

```
@protocol namespace=dev_pipeline pre_states=[]
post_states=["IR_EXTRACTED"] invalidate=["IR_STALE"]
```

这使得整个 dev_pipeline 的顺序约束（先提取 IR，再验证 Action，再验证序列，再发射代码，最后记录会话）可以被 SSG 状态机统一管理。

第五章 约束验证引擎 (Immune System)

5.1 四层语义验证层 (SVL)

Progmune 的免疫系统由四个递进的验证层组成，每一层捕获不同类别的语义异常。四层按顺序执行，前一层通过后才会进入下一层。这种分层设计使得错误可以尽早被捕获，并且每层的检查都是独立、可测试的。

SVL-1：符号存在性 (Symbol Existence)

文件： `src/validator.ts`，函数 `validateAction()`

检查内容：

- 函数名是否在 IR 中存在（或在内置白名单中）
- 动作类型 (kind) 是否合法 (call、if、for、assign、return)

检查方式：O(1) 集合查找。IR 中的所有函数名在验证开始时构建为一个 Set，每个 Action 的 function 字段在 Set 中查找。

触发条件：LLM 幻觉出不存在的函数名、输出无效的动作结构、引用了项目代码库中不存在的 API。

白名单：内置白名单包括 `console.log`、`JSON.parse` 等常用 JS 标准库函数，这些不需要在 IR 中出现也能通过 SVL-1。

SVL-2：类型完整性 (Type Integrity)

文件： `src/validator.ts`，函数 `validateAction()`

检查内容：

- 参数数量是否与 IR 签名匹配
- 参数类型是否兼容（标准化后比较：str、int、bool、dict、list 等）
- `any` 类型兼容所有其他类型（作为逃生舱）

检查方式：逐参数比较标准化类型字符串。类型标准化规则包括：将 TypeScript 类型映射到简化类型系统（string→str, number→int, boolean→bool, object→dict, Array→list），泛型参数展开为独立类型检查。

触发条件：LLM 调用函数时传入错误数量的参数、参数类型不匹配（如传入 string 但需要 number）、或者参数名与签名不符。

SVL-3：数据流有效性 (Dataflow Validity)

文件： `src/validator.ts`，函数 `checkVariableFlow()`

检查内容：

- 变量是否在引用前被声明（通过 `assignTo` 或独立的 `assign` 动作）
- 返回语句是否引用了已声明的变量
- 条件表达式是否引用了未声明的变量

检查方式：声明追踪——遍历 Action 序列，维护已声明变量的 `Map<string, string>`（变量名 → 类型）。对于每个值引用，区分字面量（字符串、数字、布尔）和变量引用（合法的 JavaScript 标识符且不在声明表中）。字面量直接通过，变量引用必须已在声明表中。

触发条件：LLM 生成的代码使用 `$变量名` 引用但目标变量尚未被任何 `call` 的 `assignTo` 或独立 `assign` 声明。这在 LLM 输出顺序混乱时常见——比如先引用了 `$token` 但生成 `$token` 的函数调用在后面。

SVL-4：协议合法性 (Protocol Legality)

文件： `src/ssg-validator.ts`

这是最复杂的一层。SVL-4 检查函数调用是否遵循协议状态机定义的顺序约束。详见第六章和第七章。

| ★ v2.2：SVL-4 在 v2.2 中从有状态的 `StateMachineValidator` 重构为纯函数 `validateTransition()`。验证逻辑不再依赖可变实例状态，而是从 `ValidationContext`（包含当前 Ledger 和派生状态）中读取上下文。这使得 SVL-4 的每次检查都是独立可审计的。

5.2 验证器的返回结构

每个验证函数返回结构化结果：

```
// validateAction 和 validateActionSequence
{ valid: boolean; errors: string[]; violations:
  ConstraintViolation[] }
```

每个 `error` 都有对应的 `ConstraintViolation`，携带完整的诊断信息（`svl`、`violatedConstraint`、`actionIndex`、`description`）。这个结构的设计使得验证结果既可以用于程序化处理（检查 `valid` 字段），也可以用于人类诊断（查看 `violations` 中的 `description`）。

5.3 批量验证流程

`validateActionSequence(actions: Action[])` 的完整流程：

1. 遍历每个 `Action`，调用 `validateAction(action, i)` —— 执行 SVL-1 和 SVL-2 检查
2. 递归处理子 `Action`（`if` 的 `thenActions/elseActions`、`for` 的 `bodyActions`）—— 子 `Action` 的索引使用父 `Action` 的索引
3. 汇总所有 `ConstraintViolation[]`
4. 如果前两层通过，执行 `checkVariableFlow()` —— SVL-3 数据流检查
5. 返回聚合的 `{ valid, errors, violations }`

注意：SVL-4（协议验证）不在此函数中执行，而是由 `planner` 在 SVL-1/2/3 通过后单独调用。这是为了支持 SSG 修复循环——如果 SVL-4 失败，`planner` 可以尝试 BFS 修复后再重新验证。

5.4 JSON Schema 预验证

文件： `src/planner.ts`，函数 `validateActionSchema()`

在 IR 感知验证之前，先进行结构完整性检查：

- 每个 `Action` 是否有 `kind` 字段
- `call` 是否有 `function`（字符串）和 `args`（数组）

- `assign` 的 `target` 是否为合法变量名
- `return` 是否在序列末尾（否则后续动作不可达）
- 变量名是否重复赋值（同一个 `assignTo` 目标被两个不同的 `call` 使用）
- `if / for` 的子 `Action` 结构递归检查

这是 P2 阶段增加的防御层，在 `JSON.parse` 成功后、IR 验证前执行，捕获格式上合法但语义结构无效的输出。例如 LLM 可能输出 `{"kind": "call"}` 但没有 `function` 字段——这在 JSON 层面是合法的，但不符合 `Action DSL` 的要求。

第六章 SSG 状态机协议

6.1 设计原理

SSG (State-Guided Generation, 状态引导生成) 是 Progmune 的核心创新。它将函数调用建模为有限状态机中的状态转移，确保 LLM 生成的代码遵循正确的 API 调用顺序。

核心思想：每个函数声明其前置状态（调用前必须满足的状态）和后置状态（调用后产生的状态），验证器维护每个命名空间的独立状态集合，在每次函数调用时检查前置条件。这类似于操作系统中系统调用的权限检查——调用者必须处于正确的状态才能执行操作。

例如，一个“生成 JWT”的函数不应该在“密码验证”之前被调用——就像你不可能在没有通过安检的情况下登机一样。SSG 状态机编码了这些顺序约束。

6.2 命名空间隔离

不同资源类型的状态机在独立的命名空间中运行：

```
_global:      UNAUTHENTICATED
auth:         UNAUTHENTICATED → PASSWORD_VERIFIED →
              TOKEN_ISSUED → SESSION_ACTIVE
file:         (empty) → FILE_OPEN → (read/write/close)
db:           (empty) → DB_CONNECTED → (query/disconnect)
```

```
dev_pipeline:  IR_STALE → IR_EXTRACTED → ACTION_VALIDATED →  
SEQUENCE_VALIDATED → CODE_EMITTED → SESSION_RECORDED
```

每个命名空间的状态变化互不影响。例如，`auth` 命名空间的 `SESSION_ACTIVE` 状态不会影响 `file` 命名空间的 `FILE_OPEN` 检查。这种隔离设计使得不同资源类型的协议可以独立演进，一个命名空间的规则变更不会破坏其他命名空间的验证逻辑。

6.3 核心数据结构

StateAnnotation（协议注解）

```
interface StateAnnotation {  
  pre_states: string[]; // 调用前必须全部满足的状态  
  post_states: string[]; // 调用后新增的状态  
  invalidate?: string[]; // 调用后失效的状态  
  namespace?: string; // 所属命名空间  
}
```

SSGRejection（协议拒绝）

```
interface SSGRejection {  
  blocked: string; // 被拦截的函数名  
  currentState: string[]; // 当前命名空间的当前状态  
  requiredState: string[]; // 被拦截函数所需的状态  
  missingFunctions: string[]; // 缺失的前置函数  
  fixPath: string[]; // BFS 找到的修复路径  
  namespace: string; // 违规所在的命名空间  
}
```

6.4 BFS 状态图搜索（确定性修复）

当函数调用违反协议时，`findFixPath()`（v2.2: `findFixPathStatic()`）在违规命名空间内执行 BFS 搜索：

```
算法：BFS 状态图搜索  
输入：namespace, currentStates, targetPreStates
```

输出：最短函数调用路径（string[]）

1. 收集命名空间内所有协议规则
2. `startState = currentStates` 的排序集合
3. 队列初始化为 [{states: startState, path: []}]
4. `visited` 集合跟踪已访问状态
5. while 队列非空：
 - a. 取出队首 {states, path}
 - b. 如果 `targetPreStates` 中的所有状态都在 `states` 中 → 返回 path
 - c. 遍历命名空间内的每个函数：
 - 如果函数的 `pre_states` 都满足 → 模拟执行
 - 计算 `nextStates`（添加 `post_states`，删除 `invalidate`）
 - 如果 `nextStates` 未访问过 → 入队
 - d. 如果 `visited > 1000` → 退出（状态爆炸保护）
6. 回退：单跳直接查找（逐个目标状态找能产出的函数）

这个算法的关键特性：

- **最优性**：BFS 保证找到最短修复路径（最小步数）
- **状态爆炸保护**：`visited` 集合上限 1000，防止在复杂状态图中无限扩展
- **回退机制**：BFS 找不到时，退化为贪心单跳匹配——逐个检查 `targetPreStates` 中的每个状态，找到能产生该状态的函数

确定性的 SSG 修复（`attemptSSGRepair()`）

当 SSG 拒绝发生时，系统不仅仅报告错误——它尝试自动修复：

1. 从 `SSGRejection.fixPath` 获取缺失函数的调用链
2. 为每个函数创建合成 Action（从 IR 推断参数类型，生成形参占位符）
3. 在被拦截的函数调用前插入修复函数
4. 重新验证修复后的序列
5. 如果仍违规且存在递归修复路径，尝试嵌套修复

只有修复成功（重新验证通过）才使用修复后的序列。这保证了确定性修复的安全性和正确性——不会因为修复而引入新的违规。

6.5 Multi-namespace 状态快照

`snapshotNamespaceStates()`（v2.2: `rebuildState()`）返回

`Record<string, string[]>`：

```
{
  "_global": ["UNAUTHENTICATED"],
  "auth": ["SESSION_ACTIVE"],
  "file": ["FILE_OPEN"],
  "dev_pipeline": ["IR_EXTRACTED", "ACTION_VALIDATED"]
}
```

每个 `StateTransition` 的 `statesBefore` 和 `statesAfter` 使用此格式，实现对每个命名空间的独立追踪。`acquired` 和 `invalidated` 字段预计算每次转移的变化（仅针对当前命名空间），供观测站直接使用而无需重新计算。

6.6 协议定义格式

`protocols.json` 是协议的声明式定义（17 条规则，4 个命名空间），支持：

```
{
  "namespaceInitialStates": {
    "_global": "UNAUTHENTICATED",
    "auth": "UNAUTHENTICATED",
    "dev_pipeline": "IR_STALE"
  },
  "rules": {
    "verify_password": {
      "namespace": "auth",
      "pre_states": ["UNAUTHENTICATED"],
      "post_states": ["PASSWORD_VERIFIED"]
    }
  }
}
```

协议规则也可以从代码中的 `JSDoc @protocol` 注解提取。两种来源合并时，IR 中的 `@protocol` 注解优先，但 JSON 中的 `namespace` 定义作为权威命名空间声明。

第七章 Semantic Ledger Runtime

★ v2.2 核心新增

7.1 设计原理

v2.1 中，`StateMachineValidator` 实例维护可变状态（`Map<string, Set<string>>`），`StateTransition[]` 是被动日志。验证决策来自可变对象，而非不可变的事件记录。

v2.2 反转这一关系：**`StateTransition[]` 成为系统唯一真相源（Single Source of Truth）**，命名空间状态从中派生。

核心洞察：Event Sourcing 不仅使系统可审计——它将验证逻辑从有状态对象中解放出来，变为可组合、可测试、可在任何上下文中独立验证的纯函数。一个 `validateTransition()` 调用不依赖任何外部可变状态——给它相同的输入，它永远返回相同的输出。

7.2 纯函数架构（Pure Function Architecture）

v2.2 将所有验证逻辑提取为 16 个纯函数，消除对 `StateMachineValidator` 可变实例的依赖。其中 5 个核心函数构成了验证引擎的主体：

纯函数	签名	职责
<code>rebuildState()</code>	<code>(ledger, nsInit?) → Record<string, string[]></code>	从 Ledger 折叠重建 per-namespace 状态快照。遍历整个 Ledger，对每条有效转移应用其 <code>acquired/</code>

纯函数	签名	职责
		invalidated 增量。无效转移被跳过。
applyTransitionDelta()	(stateMap, transition) → void	将单条转移的增量应用到可变状态映射。用于增量更新 (O(1) per transition)，避免在循环中反复调用 rebuildState()。
validateTransition()	(ctx, fn, idx, rules, nsInit, hash) → {valid, transition, rejection?}	纯函数验证单条转移，无副作用。检查 ctx.currentState 是否满足候选函数的 pre_states，构建 statesBefore/ statesAfter，运行 Invariant-1 自检。
checkLedgerConsistency()	(ledger, nsInit?) → {consistent, violations[]}	单遍 O(n) 检查整个 Ledger 的 Invariant-0 + Invariant-1。使用 applyTransitionDelta() 增量维护预期状态，不与 rebuildState() 重复计算。
findFixPathStatic()	(rules, ns, current, target) → string[]	BFS 静态搜索修复路径。不依赖任何实例状态，所有输入显式传递。

其余 11 个纯函数覆盖完整性、查询和诊断： hashRules() 、 hashLedger() 、 diffLedgers() 、 findProducer() 、 findConsumer() 、

`findViolations()`、`findTransition()`、`listAllStates()`、`explainRejection()`、`rejectionToJSON()`、`toSnapshot()`。

ValidationContext — $O(n)$ 循环模式

v2.1 在验证循环中每次调用 `rebuildState()` ($O(n)$ per iteration)，导致总复杂度 $O(n^2)$ 。v2.2 引入 `ValidationContext` 预计算状态，降为 $O(n)$ ：

```
interface ValidationContext {
    ledger: StateTransition[];
    currentState: Record<string, string[]>;
}

//  $O(n)$  验证循环 — 每次迭代  $O(1)$  状态更新:
ctx = { ledger: [], currentState: rebuildState([], nsInit) }
for each action:
    { valid, transition } = validateTransition(ctx, fn, i,
        rules, nsStates, ruleHash)
    ctx.ledger.push(transition)
    ctx.currentState = transition.statesAfter //  $O(1)$  增量更新
```

向后兼容： `StateMachineValidator` 类保留，内部全部委托给纯函数：

- `apply()` → 委托给 `validateTransition()`
- `applyWithTransition()` → 委托给 `validateTransition()`
- `snapshotNamespaceStates()` → 委托给 `rebuildState(this.ledger)`
- `getTrace()` → 返回 `this.ledger`

生产代码 (`planner`、`semantic-trace`、`check`、`p0_ssg_demo`) 全部直接使用纯函数，零 `StateMachineValidator` 实例化。

7.3 不变式系统 (Invariant System)

v2.2 引入双重不变式，为每条 `StateTransition` 提供类似 DNA 聚合酶校对的一致性保证。两个不变式互补——Invariant-0 保证转移与历史一致，Invariant-1 保证转移内部自洽。

Invariant-0 — Before Consistency (前置一致性)

`transition.statesBefore` $\equiv$ `rebuildState(ledger[0..i-1])`

转移执行前的状态快照必须等于从历史 Ledger 重建的状态。

这个不变式回答的问题是："转移声称自己是在某个状态下执行的，这个声称与历史记录一致吗？"如果一条转移声称自己是在 `auth: [TOKEN_ISSUED]` 状态下执行的，但从历史 Ledger 重建出来的状态是 `auth: [PASSWORD_VERIFIED]`，那么 Invariant-0 就会标记违规。这意味着要么转移的 `statesBefore` 被错误记录，要么历史 Ledger 缺失了某些转移。

Invariant-1 — Delta Consistency (增量一致性)

`transition.statesAfter` $\equiv$ `applyDelta(statesBefore, acquired, invalidated)`

转移执行后的状态快照必须等于对执行前状态应用增量的结果。

这个不变式回答的问题是："转移声称自己产生的效果，与它记录的增量变化一致吗？"Invariant-1 捕获 Invariant-0 漏掉的 内部不一致 转移——即 `statesBefore` 与历史一致但 `statesAfter` 与自身的 `acquired/invalidated` 不符的转移：

```
// 声称获取 TOKEN_ISSUED, 但 statesAfter 中没有 — Invariant-1 会捕获
{
  "statesBefore": {"auth": ["PASSWORD_VERIFIED"]},
  "acquired": ["TOKEN_ISSUED"],
  "invalidated": [],
  "statesAfter": {"auth": ["PASSWORD_VERIFIED"]} // ←
               TOKEN_ISSUED 缺失!
}
```

Invariant-1 是关键的安全网：即使历史账本完全一致，单条转移内部的记录错误（如 `acquired` 和 `statesAfter` 不匹配）也会被捕获。这种错误可能由序列化 bug、并发写入问题或手动编辑会话 JSON 文件引入。

不变式可视化

Ledger: [T0, T1, T2, ... Ti]

Invariant-0 校验:

Ti.statesBefore
≡
rebuildState([T0.. Ti-1])
→ 历史一致性

Invariant-1 校验:

Ti.statesBefore
+ acquired
- invalidated
≡
Ti.statesAfter
→ 增量自洽性

checkLedgerConsistency() 算法

单个 O(n) 遍历完成两个不变式的检查:

算法: checkLedgerConsistency(ledger, nsInit)

输入: StateTransition[], Map<string, string>

输出: { consistent: boolean, violations:

LedgerConsistencyViolation[] }

1. 初始化 stateMap = fromSnapshot(rebuildState([], nsInit))
2. 初始化 violations = []
3. for i = 0; i < ledger.length; i++:
 - a. T = ledger[i]
 - b. // Invariant-0: statesBefore 是否等于当前 stateMap?
 - c. expected = toSnapshot(stateMap)
 - d. if JSON.stringify(norm(T.statesBefore)) !== JSON.stringify(norm(expected)):
violations.push({ index: i, invariant: "before-consistency", expected, actual: T.statesBefore })
 - e. // Invariant-1: statesAfter 是否等于
applyDelta(statesBefore, acquired, invalidated)?
 - f. expectedAfter = applyDelta(T.statesBefore, T.acquired, T.invalidated)
 - g. if JSON.stringify(norm(T.statesAfter)) !== JSON.stringify(norm(expectedAfter)):
violations.push({ index: i, invariant: "delta-consistency", expected: expectedAfter, actual: T.statesAfter })
 - h. // 更新 stateMap (仅当转移有效时)

```
i. if T.valid: applyTransitionDelta(stateMap, T)
4. return { consistent: violations.length === 0, violations }
```

关键实现细节：使用 `applyTransitionDelta()` 增量更新 `stateMap`，而不是在循环内调用 `rebuildState()`（那会导致 $O(n^2)$ ）。两个不变式在同一个循环中完成，不需要额外的遍历。

7.4 约束快照 (Constraint Snapshot)

`hashRules(rules: Map<string, StateAnnotation>): string` 对协议规则集进行确定性 SHA256 哈希。规则按函数名排序，每个规则的字段按固定顺序序列化，确保相同规则集永远产生相同的哈希：

```
Transition @ 2026: ruleHash = "7d8f9a..."
2027 年规则变更 → ruleHash = "a1b2c3..."
回放 2026 Ledger → ruleHash 不匹配
→ "确定性回放不可行：约束集已变更"
```

`ruleHash` 在会话初始化时由 `planner.ts` 计算一次，然后注入到每条 `StateTransition`、每个 `Attempt` 和 `ExecutionSession` 中。这个不变式保证了从单条转移到完整会话的约束谱系一致性。

7.5 账本完整性指纹 (Ledger Integrity Fingerprint)

`hashLedger(ledger: StateTransition[]): string` 对整个 Ledger 进行确定性 SHA256 哈希：

```
// 规范化排序后哈希，确保确定性
hashLedger(ledger) → "b44f80a2b4d1d1f6"
```

每条转移的 `statesBefore` 和 `statesAfter` 在哈希前按命名空间排序，确保 JSON 序列化的确定性。这个指纹是一个防篡改的完整性证明——如果 Ledger 文件被手动修改（即使是一个空格），指纹也会改变。`check.ts` 在每次健康检查时输出当前全局账本的完整性指纹。

7.6 Ledger Query API

v2.2 将账本转化为可查询数据库，提供 5 个纯查询函数：

查询函数	签名	用途
<code>findProducer(state, L)</code>	<code>(state: string, ledger: StateTransition[]) → TransitionMatch[]</code>	查找哪些转移产生了指定状态（按 <code>acquired</code> 匹配）
<code>findConsumer(state, L)</code>	<code>(state: string, ledger: StateTransition[]) → TransitionMatch[]</code>	查找哪些转移以指定状态为前置条件
<code>findViolations(L)</code>	<code>(ledger: StateTransition[]) → StateTransition[]</code>	返回所有 <code>invalid</code> 转移
<code>findTransition(idx, L)</code>	<code>(index: number, ledger: StateTransition[]) → StateTransition null</code>	按 <code>actionIndex</code> 查找转移
<code>listAllStates(L)</code>	<code>(ledger: StateTransition[]) → {ns: string, state: string}[]</code>	列出账本中出现的所有 (namespace, state) 对

这些函数是纯函数——它们不依赖任何外部状态或可变对象。给定相同的 `Ledger`，永远返回相同的结果。这为终端观测站（`semantic-trace.ts`）的 `--query` 和 `--query-all` 命令提供了底层支持。

7.7 账本差异比较 (Ledger Diff)

`diffLedgers(ledgerA, ledgerB): LedgerDiff` 比较两个账本，返回结构化差异报告：

```
interface LedgerDiff {  
  unchanged: number; // 两个账本中 hash 完全相同的转移数  
  onlyInA: number; // 仅在 A 中出现的转移数  
  onlyInB: number; // 仅在 B 中出现的转移数  
  changed: number; // 索引相同但 hash 不同的转移数  
}
```

```
Ledger Diff: A (2 transitions) vs B (3 transitions)  
├─ Unchanged: 1    ← 转移 hash 完全相同  
├─ Only in A: 0    ← 仅在 A 中出现的  
├─ Only in B: 1    ← 仅在 B 中出现的  
└─ Changed: 1      ← 索引相同但 hash 不同
```

Ledger Diff 的用途包括：比较同一意图的两次不同执行（看 LLM 输出如何变化）、比较修复前后的账本（看 SSG 修复插入了什么）、比较规则变更前后的回放结果（检测约束变更的影响）。终端可通过 `semantic-trace.ts --diff-ledgers <A> <B>` 使用。

7.8 三大支柱：可重放 → 可查询 → 可证明



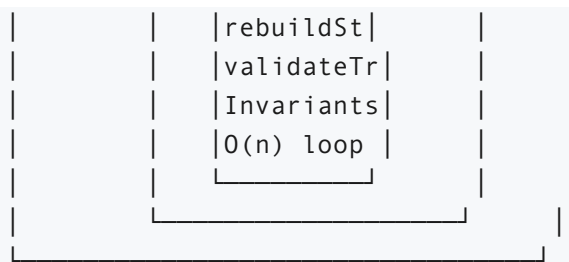


图 7.1: Semantic Ledger Runtime 三大支柱——从核心到外延的能力递进

这三大支柱是递进关系：

1. **Replayable (可重放)** 是基础——任何人都可以从空状态出发重放 Ledger 得到相同的状态。没有这个能力，后续的查询和证明都无从谈起。
2. **Queryable (可查询)** 建立在可重放之上——既然状态可以从 Ledger 确定性派生，我们就可以在 Ledger 上运行结构化查询。
findProducer/Consumer 回答"谁产生了/消费了某个状态";
findViolations 回答"哪些转移有问题"。
3. **Provable (可证明)** 是最外层——hashLedger 提供防篡改的完整性证明，ruleHash 提供跨版本的约束谱系追溯，diffLedgers 提供审计级的差异比较。

7.9 实验验证

对 6 个真实执行会话（11 条转移：7 有效，4 无效）进行完整检查：

- **全部通过** Invariant-0 + Invariant-1 + Replay 三重验证。所有 6 个会话的不变式检查均返回 `consistent: true`。
- **完整性指纹**：2f701ec86643dc63 （全局账本的 SHA256 指纹，在 check.ts Step 4 中计算和输出）
- **Invariant-1 负向测试**：手工构造内部不一致转移
(acquired=["TOKEN_ISSUED"], statesAfter 中无 TOKEN_ISSUED) → 正确检出 1 个 delta-consistency 违规，验证了不变式系统的故障检测能力
- **纯函数验证**：p0_ssg_demo.ts 覆盖 7 个端到端场景（含 auth 协议完整流程、dev_pipeline 5 阶段验证、协议违规拦截、Invariant 负向测试、Query API 演示），零 StateMachineValidator 实例化

第八章 规划器 (Planner) | ★ v2.2 更新

8.1 核心规划循环

`plan(userIntent: string): Promise<Action[]>` 是系统的核心入口，位于 `src/planner.ts` (~920 行 | ★ v2.2 更新)。

规划流程

1. 初始化
 - └─ 加载 IR (`ir.json`)
 - └─ 创建 `ExecutionSession { sessionId, intent, attempts: [] }`
 - └─ `hashRules()` 计算 `ruleHash` ★ v2.2
 - └─ 创建 `IRSnapshot` (用于确定性回放)
 - └─ 估算 `prompt tokens`
2. 快速通道检查
 - └─ 语义模板缓存 (`memory-layer`)
 - └─ 命中 & `successRate ≥ 0.8` & `useCount ≥ 2` → 直接返回
 - └─ 抗体免疫查询 (`failure-corpus`)
 - └─ ACL-4: 0 LLM 调用, 直接构建 `Action[]`
 - └─ ACL-3: 注入修复路径提示到 `prompt`
3. 函数排序与提示构建
 - └─ Jaccard 相似度排序 (意图关键词 vs 函数名)
 - └─ 取 top-15 函数
 - └─ 构建紧凑函数列表
4. 检查点恢复 (如果存在之前的检查点)
5. 主重试循环 (`maxRetries = 3`)
 - └─ LLM 调用 (`chat` 或 `generate`, `temperature=0.0`)
 - └─ `parseActionJSON()` 解析输出
 - └─ `validateActionSchema()` JSON 结构预验证
 - └─ `enrichActions()` 补充 IR 元数据
 - └─ `validateActionSequence()` SVL-1/2/3 序列验证
 - └─ ★ v2.2 `validateTransition()` 纯函数 SVL-4 验证 (使用 `ValidationContext`)
 - └─ `attemptSSGRepair()` 确定性修复
 - └─ `checkSemantic()` 语义合约验证
6. ★ v2.2 `checkLedgerConsistency()` 不变式验证
 - └─ 对整个 `Attempt` 的 `Ledger` 运行 `Invariant-0 + Invariant-1`

7. 失败降级

└─ generateFallbackPlan() 本地规则回退

★ **v2.2 关键变更**：步骤 5 中的 SVL-4 验证不再实例化

StateMachineValidator。而是使用 ValidationContext 模式：

```
const ctx: ValidationContext = { ledger: [], currentState:
rebuildState([], nsInit) };
for (let i = 0; i < actions.length; i++) {
  const { valid, transition, rejection } =
validateTransition(ctx, fn, i, rules, nsStates, ruleHash);
  ctx.ledger.push(transition);
  if (valid) ctx.currentState = transition.statesAfter;
  // 处理违规...
}
// 循环结束后：
checkLedgerConsistency(ctx.ledger, nsInit); // 不变式验证
```

8.2 LLM 交互设计

系统提示 (System Prompt)

你是程序合成助手。只输出 JSON 数组，不输出解释。

格式（紧凑一行）：

```
[{"f": "函数名", "to": "变量名", "a": [{"n": "参数名", "t": "类型", "v":
值}]}, {"r": "变量名"}]
```

铁律：只输出 JSON 数组，双引号，不输出解释

关键设计决策：

- **System/User 分离**：静态规则放 system prompt，动态内容（函数列表、意图）放 user prompt。利用各 LLM 平台的 system prompt 缓存机制降低延迟和成本
- **温度 0.0**：所有 LLM 调用使用 `temperature: 0.0`，最大化确定性
- **紧凑 JSON**：单字母键名减少 token 消耗（f=function, to=assignTo, a=args, n=name, t=type, v=value, r=return）

提示构建策略

- 从 IR 中选出与意图关键词 Jaccard 相似度最高的 top-15 函数
- 使用 `buildCompactFuncList()` 生成紧凑格式：
`fnName(p1:type,p2:type)->returnType`
- 如果检测到意图中提到的函数名已在 IR 中存在，将其加入禁止列表（防止 LLM 尝试"实现"已有函数）
- 重试时在 prompt 中注入具体错误信息（来自上轮验证失败的 `ConstraintViolation[]`）

8.3 免疫加速机制

语义模板缓存

`findSemanticTemplate(userIntent)` 检查 `memory-layer` 中是否有高置信度（`successRate` ≥ 0.8 , 使用 ≥ 2 次）的模板匹配。命中后直接返回缓存的 `Action[]`，跳过 LLM 调用。

抗体推理

`queryAntibodies(userIntent, minACL)` 查询免疫记忆：

1. 收集所有失败模式及其修复路径
2. 按 ACL 等级过滤（默认 ACL-3+）
3. 计算当前意图与历史意图的 Jaccard 相似度（基于词语重叠）
4. 过滤相似度 ≥ 0.2 的结果
5. 按相似度降序排序

ACL-4（全局稳定抗体）：修复路径在 ≥ 10 次出现且跨 ≥ 5 个不同意图。触发时完全跳过 LLM，直接从修复路径构建 `Action[]`。

ACL-3（跨任务验证抗体）：出现在 ≥ 4 次或跨 ≥ 3 个不同意图。将修复路径作为提示注入 LLM prompt，帮助 LLM 生成正确的调用顺序。

8.4 规划器检查点

`saveCheckpoint()` / `loadCheckpoint()` 实现执行持久化：

```
interface PlannerCheckpoint {
  intent: string;
  attemptIndex: number;           // 已完成的尝试次数
  sessionAttempts: Attempt[];    // 序列化的尝试记录
  currentPrompt: string;         // 当前重试 prompt
  useSystem: boolean;            // 是否使用 system prompt
  timestamp: string;
}
```

检查点用 `intent` 的 `base64 hash` 作为文件名，存储于 `.progmune_corpus/checkpoints/`。在 LLM 调用失败或进程重启后可从检查点恢复。

8.5 降级策略

当 LLM 3 次重试全部失败时，系统进入本地规则回退：

`generateFallbackPlan(intent, ir)` 从意图中提取关键词，匹配 IR 中名称包含这些关键词的函数，生成简单的顺序调用 `Action[]`。这个回退方案保证了系统在任何情况下都能产生输出（尽管质量可能不如 LLM 生成的方案）。

第九章 免疫记忆与抗体推理

9.1 失败语料库 (Failure Corpus)

文件： `src/failure-corpus.ts` (~650 行)

失败语料库是 Progmune 的“免疫记忆”——存储每次约束违规的详细记录，支持模式挖掘、统计分析和抗体生成。就像生物免疫系统在感染后保留记忆 B 细胞以便快速应对相同的病原体，Progmune 的失败语料库在每次约束违规后保留完整的上下文，以便未来遇到相似意图时加速修复。

数据存储

```
.progmune_corpus/
├── .seq                # 全局递增序列号（保证跨进程唯一 ID）
├── sessions/          # 执行会话（ExecutionSession JSON）
│   └── sess_*.json
├── checkpoints/       # 规划器检查点
│   └── ckpt_*.json
└── YYYY-MM-DD/        # 按日期分区的失败记录
    └── fail_*.json
```

FailureRecord 结构

```
interface FailureRecord {
    id: string;                // fail_<timestamp>_<seq>
    timestamp: string;         // ISO 8601
    intent: string;            // 用户意图
    projectFunctions: string[]; // IR 中的全部函数名
    violatedSVL: SVL;          // 违规层级
    constraintType: string;     // 具体约束类型
    actionSequence: any[];      // 导致违规的动作序列
    errorDetail: string;        // 详细错误信息
    ssgState?: string[];        // 违规时的 SSG 状态
    ssgTrace?: { ... }[];      // SSG 完整跟踪
    ssgFixPath?: string[];      // SSG 修复路径
    ssgMissingFunctions?: string[]; // 缺失的前置函数
    plannerAttempt: number;      // 第几次尝试
    plannerRetryTotal: number;   // 总重试次数
}
```

9.2 会话归一化

`normalizeSession()` 检测新旧两种会话格式：

- **新格式** (ExecutionSession) : `attempts[0].violations` 存在
- **旧格式** (IntentSession) : `attempts[0].violatedSVL` 存在

旧格式通过 `convertOldSession()` 上转为 `ExecutionSession`，确保所有下游消费者看到统一的接口。这个归一化层使得系统可以无缝处理 v2.0、v2.1 和 v2.2 产生的会话数据。

9.3 失败基因组 (Failure Genome)

`getFailureGenome()` 对所有会话进行统计分析：

指标	说明
<code>totalFailures</code>	总违规次数
<code>bySVL</code>	按 SVL 层级的分布 ({ "SVL-1": n, "SVL-2": m, ... })
<code>byConstraintType</code>	按约束类型的分布 (如 "function_not_found": n)
<code>topPatterns</code>	最常见的失效模式 (SVL:constraintType 组合, 按频次排序)
<code>commonFixPaths</code>	最常用的修复路径 (按使用次数排序, 含 count 和 fixPath)
<code>averageRetriesToSuccess</code>	平均成功所需重试次数

9.4 抗体推理引擎

ACL 等级体系

ACL-1 (单例观察):	<code>count ≥ 1</code>
ACL-2 (重复观察):	<code>count ≥ 2</code>
ACL-3 (跨任务验证):	<code>count ≥ 4</code> 或 <code>distinctIntents ≥ 3</code>
ACL-4 (全局稳定):	<code>count ≥ 10</code> 且 <code>distinctIntents ≥ 5</code>

阈值可通过环境变量配置：

- `PROGMUNE_ACL4_COUNT` (默认 10)
- `PROGMUNE_ACL4_INTENTS` (默认 5)

- `PROGMUNE_ACL3_COUNT` (默认 4)
- `PROGMUNE_ACL3_INTENTS` (默认 3)
- `PROGMUNE_ACL2_COUNT` (默认 2)

学习模式 (LearnedPattern)

```
interface LearnedPattern {
    signature: string;           //
                                "SVL-4:missingFunc1,missingFunc2"
    violation: string;           // 人类可读的违规描述
    fixPath: string[];           // 修复路径
    occurrenceCount: number;     // 出现次数
    distinctIntents: string[];   // 涉及的不同意图
    resolvedRate: number;        // 修复成功率
    antibodyLevel: AntibodyLevel;
    firstSeen: string;           // 首次出现时间
    lastSeen: string;            // 最近出现时间
}
```

查询匹配算法

`queryAntibodies(intent, minACL)` 的匹配逻辑:

1. 过滤 ACL 等级 $\geq$ minACL 的模式
2. 过滤有修复路径的模式 (无修复路径的抗体无法用于加速)
3. 对每个模式计算与当前意图的 Jaccard 相似度 (基于词语重叠)
4. 过滤相似度 ≥ 0.2 的模式
5. 按相似度降序排序, 返回最佳匹配

9.5 语义热力图

`getSemanticHeatmap()` 提供四维分析:

1. **fragileProtocols**: 最常被违规协议拦截的函数 (按拦截次数排序)
2. **svlHotspots**: 各 SVL 层级的热度分布 (百分比), 标识哪个验证层最常被触发
3. **constraintClusters**: 同一会话中多种约束类型共现的集群, 揭示相关的失效模式

4. **highFrictionIntents**: 需要最多适配次数的意图任务，标识系统的高摩擦点

9.6 抗体效能统计

`getAntibodyStats()` 量化免疫系统的实际效益:

指标	说明
<code>totalHits</code>	抗体总命中数
<code>fastPathHits</code>	ACL-4 快速通道命中 (0 LLM 调用)
<code>injectedHintHits</code>	ACL-3 提示注入命中
<code>totalLLMCallsSaved</code>	节省的 LLM 调用次数
<code>totalTokensSaved</code>	估算节省的 token 数
<code>byLevel</code>	按 ACL 等级的命中分布
<code>topSignatures</code>	最常见的抗体签名

第十章 代码发射

10.1 设计原理

当 LLM 输出的 Action 序列通过所有四层验证和不变式检查后, `emitter.ts` 将其编译为可执行的 TypeScript 代码。发射器是系统中唯一的"写"操作——将经过验证的语义结构转化为可编译的代码文本。

发射器的设计原则是"保守翻译": 只做直接的语法映射, 不在发射过程中添加任何业务逻辑或优化。这样可以保证发射输出的行为与验证通过的 Action 序列完全一致。

10.2 发射流程

```
Action[] → 遍历 → 类型分发 → emitCall | emitAssign | emitReturn |  
emitIf | emitFor → TypeScript 代码字符串
```

- **call**: 生成 `const varName = functionName(arg1, arg2);` (有 `assignTo`) 或 `functionName(arg1, arg2);` (无 `assignTo`)
- **assign**: 生成 `target = value;` (`value` 为字面量时直接输出, 为嵌套 Action 时递归发射)
- **return**: 生成 `return value;` (`value` 处理同 `assign`)
- **if**: 生成 `if (condition) { ... } else { ... },` `thenActions` 和 `elseActions` 各自递归发射
- **for**: 生成 `for (const variable of iterable) { ... },` `bodyActions` 递归发射

发射器还负责合理的缩进 (4 空格) 和换行, 使输出的 TypeScript 代码可读。变量名引用 (以 `$` 为前缀的 JSON wire format) 在发射时去掉 `$` 前缀。

10.3 Action 运行时沙箱

`action-runtime.ts` 提供 `executeActionCode(code: string):`

`Action[]`, 将自由格式的 DSL 代码解析为 `Action[]`。这作为 LLM JSON 输出解析失败时的降级路径——如果 LLM 无法生成符合 JSON wire format 的输出, 它可以生成更自由的 DSL 格式。

沙箱使用 `new Function()` 构造受限执行环境, 暴露 `call`、`callAssign`、`ifBlock`、`ifElse`、`assign`、`output` 六个 API 函数。变量通过 Proxy 对象管理, 确保沙箱内的变量不会泄漏到外部作用域。

第十一章 观测站系统 | ★ v2.2 大幅扩展

11.1 终端观测站 (semantic-trace.ts)

文件: `src/semantic-trace.ts` (~1800 行 | ★ v2.2 扩展)

提供丰富的终端 UI, 支持以下命令:

命令	功能	v2.2
<code>ts-node src/semantic-trace.ts</code>	所有会话摘要 (ID、意图、Attempt 数、成功率)	
<code>ts-node src/semantic-trace.ts <id></code>	单会话完整时间线 (含每轮尝试的转移详情)	
<code>ts-node src/semantic-trace.ts replay <id></code>	逐步认知回放 (动画展示每次尝试的决策过程)	
<code>ts-node src/semantic-trace.ts --states <id></code>	状态转移可视化 (per-namespace 的状态变化图)	
<code>ts-node src/semantic-trace.ts --validate <id></code>	确定性回放验证 (重建状态 + Invariant 检查)	★ 扩展为纯函数实现
<code>ts-node src/semantic-trace.ts --ledger <id></code>	单会话 Ledger 深度验证 (Invariant-0 + Invariant-1 + Replay + 指纹)	★ 新增
		★ 新增

命令	功能	v2.2
<code>ts-node src/semantic-trace.ts --query <id> <op></code>	单账本查询 (producer/consumer/violations/transition/states)	
<code>ts-node src/semantic-trace.ts --query-all <op> <state></code>	跨会话聚合查询 (如查找所有产生 TOKEN_ISSUED 的转移)	★ 新增
<code>ts-node src/semantic-trace.ts --stats</code>	语料库级别账本统计 (总会话数、总转移数、有效/无效比例等)	★ 新增
<code>ts-node src/semantic-trace.ts --diff-ledgers <A> </code>	跨会话账本差异比较 (新增/删除/变更/不变转移数)	★ 新增
<code>ts-node src/semantic-trace.ts --genome</code>	失败基因组摘要 (按 SVL/约束类型的分布)	
<code>ts-node src/semantic-trace.ts --learned</code>	抗体注册表 (含 ACL 等级、签名、成功率)	
<code>ts-node src/semantic-trace.ts --antibodies</code>	抗体效能指标 (命中数、LLM 节省、token 节省)	
<code>ts-node src/semantic-trace.ts --heatmap</code>	语义热力图 (脆弱协议、SVL 热点、约束集群、高摩擦意图)	
	IR 快照列表	

命令	功能	v2.2
<code>ts-node src/semantic-trace.ts --snapshots</code>		
<code>ts-node src/semantic-trace.ts --diff <A> </code>	IR 快照差异对比（函数新增/删除/修改）	

认知回放 (Cognitive Replay)

`replaySession()` 以动画形式逐步展示规划器的每一次尝试:

- 1. 显示尝试序号和总次数 (如"尝试 2/3")
- 2. 列出 LLM 生成的动作序列
- 3. 展示检测到的语义异常及所属免疫层 (SVL-1/2/3/4)
- 4. 展示免疫响应 (修复路径激活 / 适配请求)
- 5. 对比前后尝试的差异 (`persisted/dropped/acquired` 状态)
- 6. 显示进度条
- 7. 最终展示成功路径和状态机跟踪

11.2 Web 仪表盘 (obs-web.ts)

文件: `src/obs-web.ts` (~365 行)

零依赖 HTTP 服务器 (使用 Node 内置 `http` 模块), 运行于端口 3100。

API 端点

端点	返回
<code>GET /api/sessions</code>	所有会话列表 (含 ID、意图、状态、时间)
<code>GET /api/sessions/:id</code>	单会话详细信息 (含所有 <code>Attempt</code> 和 <code>Transition</code>)

端点	返回
<code>GET /api/genome</code>	失败基因组统计
<code>GET /api/heatmap</code>	语义热力图数据
<code>GET /api/antibodies</code>	抗体效能统计
<code>GET /api/learned</code>	学习模式注册表

前端界面

单页应用（inline HTML/CSS/JS），包含五个选项卡：

- **Overview**：总览面板——会话数、违规数、抗体命中率、平均重试次数
- **Sessions**：会话列表，点击查看完整时间线（含 transitions 和 violations 的详细卡片）
- **Genome**：失败基因组可视化——按 SVL 和约束类型的分组柱状图
- **Heatmap**：语义热力图——高摩擦意图和脆弱协议的交互式表格
- **Antibodies**：抗体效能——ACL 分布饼图和 Top 10 签名

暗色主题，GitHub 风格颜色方案。数据通过 fetch API 获取，页面无刷新切换。

11.3 免疫健康检查（check.ts） | ★ v2.2 更新

文件： `src/check.ts` （~380 行 | ★ v2.2 扩展）

`npm run check` 一键执行六项健康检查：

步骤	检查内容	方式	v2.2
1/6	IR 重新提取	<code>npx ts-node src/extract-ir.ts .</code>	
2/6		<code>npx tsc --noEmit</code>	

步骤	检查内容	方式	v2.2
	TypeScript 类型检查		
3/6	SSG dev_pipeline 协议验证	纯函数 validateTransition() 循环（5 阶段 pipeline）	★ 纯函数化（零 StateMachineValidator 实例化）
4/6	Ledger 不变 量 + 回放 + 完整性指纹	Invariant-0 + Invariant-1 + Replay + hashLedger()	★ 新增
5/6	失败基因组概 要	getFailureGenome()	
6/6	抗体效能统计	getAntibodyStats()	

新增 `--ledger <sessionId>` 参数支持单会话 Ledger 深度验证（per-attempt 转移详情 + 一致性 + 回放）。彩色 ANSI 输出，退出码 1 表示存在失败。

第十二章 MCP 服务器

12.1 设计

`mcp-server.mts` 将 Progmune Runtime 暴露为 MCP（Model Context Protocol）工具服务器，使任何支持 MCP 的 LLM 客户端（Claude Desktop、VS Code、Cline 等）可以调用 Progmune 的约束验证能力。

MCP 服务器是系统的"对外接口层"。它将内部的规划、验证、发射流程封装为 6 个标准化工具，每个工具有明确的输入/输出 JSON Schema。

12.2 暴露的工具

工具名	功能
<code>progmune_plan</code>	调用规划器，从自然语言意图生成 Action 序列。输入：intent 字符串。输出：Action[] 和生成的 TypeScript 代码
<code>progmune_validate</code>	验证给定的 Action 序列（JSON wire format）。返回验证结果（valid + violations）
<code>progmune_emit</code>	将 Action 序列编译为目标代码（TypeScript）。返回编译后的代码字符串
<code>progmune_ir_extract</code>	从项目根提取 IR，重新生成 ir.json。返回提取的函数数量和外部函数数量
<code>progmune_observatory</code>	查询观测站数据（会话列表、基因组、抗体统计）。返回请求的数据类型
<code>progmune_check</code>	运行免疫健康检查（6 步 pipeline）。返回检查结果和免疫状态总结

12.3 多项目隔离

通过 `PROGMUNE_PROJECT_DIR` 环境变量支持多项目隔离：

- 每个项目有独立的 `.progmune_corpus/`（会话、失败记录、检查点）
- 每个项目有独立的 `ir.json`（函数签名）
- MCP 服务器在调用工具时设置此变量，确保工具在正确的项目上下文中执行

第十三章 关键辅助系统

13.1 LLM 抽象层

文件: `src/llm.ts`

支持多个 LLM 提供商:

提供商	默认模型	配置方式
DeepSeek	deepseek-chat	默认 (无需额外配置)
OpenAI	gpt-4	<code>LLM_PROVIDER=openai</code>
Ollama	llama3	<code>LLM_PROVIDER=ollama</code>

环境变量: `LLM_API_KEY`、`LLM_BASE_URL`、`LLM_MODEL`、`LLM_PROVIDER`。

Token 估算: CJK 字符 ~1.5 token/字, 其他 ~0.4 token/字符。用于在构建 prompt 前估算 token 消耗, 防止超出模型上下文窗口。

13.2 语义快照

文件: `src/semantic-snapshot.ts`

`createSnapshot(ir, intent)` 在每次规划会话开始时创建 IR 的快照 (`IRSnapshot`), 包含完整函数签名和时间戳。快照支持:

- 确定性回放 (验证历史会话在当时 IR 下是否合法——即使当前 IR 已经变化)
- 差异对比 (`diffSnapshots(a, b)` 显示函数的新增/删除/修改)
- 回归检测 (会话中引用的函数是否在后续 IR 版本中消失——如果消失, 说明代码库发生了 `breaking change`)

13.3 记忆层

文件： `src/memory-layer.ts`

基于语义模板的记忆系统：

- `findSemanticTemplate(intent)`：匹配历史成功模板，返回最相似的 `Action[]`
- `recordEpisode({intent, actions, success})`：记录每次规划结果，更新模板库
- 维护使用次数和成功率统计（ $\text{successRate} = \text{成功次数} / \text{总使用次数}$ ）

13.4 反馈系统

文件： `src/feedback.ts`

追踪每个函数在成功/失败会话中的使用频率，计算成功率：

- `getFunctionSuccessRate(functionName)`：返回 0-1 成功率
- 用于规划器中的函数排序和 LLM prompt 优化——高成功率函数在 top-15 排序中获得加分
- 函数在失败会话中出现频次过高时，其权重被降低

13.5 工具模块

文件	功能
<code>utils.ts</code>	Jaccard 相似度（用于意图匹配和抗体查询）、关键词提取（中英文分词 + 函数名驼峰分词）
<code>file-lock.ts</code>	基于文件的并发锁（通过 lock 文件确保 corpus 写入的原子性，防止多进程竞争）
<code>search-planner.ts</code>	基于搜索的备用规划器（无需 LLM，直接在 IR 中搜索匹配函数组合）

文件	功能
<code>immune-reporter.ts</code>	免疫报告生成（将 Failure Genome 和 Antibody Stats 格式化为可打印报告）

第十四章 数据流总览 | ★ v2.2 更新

14.1 完整交互序列

1. 用户输入意图: "实现用户登录功能"
2. MCP Server 接收 → 调用 `plan("实现用户登录功能")`
3. Planner 初始化:
 - └─ 加载 `ir.json` (IR 提取的函数签名列表)
 - └─ 创建 `ExecutionSession { sessionId, intent, attempts: [] }`
 - └─ `hashRules()` 计算 `ruleHash` → "7d8f9a..." ★ v2.2
 - └─ 创建 `IRSnapshot` (用于确定性回放)
 - └─ 检查语义模板缓存 → 未命中
 - └─ 检查抗体 → 命中 ACL-3 抗体
 - └─ 注入提示: "已知正确调用顺序: `verify_password` → `generate_jwt` → `create_session`"
4. Planner 构建 LLM Prompt:
 - └─ System: 静态规则 (JSON 格式要求)
 - └─ User: top-15 相关函数 + 意图 + 抗体提示
 - └─ 估算 tokens: ~1200
5. LLM 返回 JSON:

```
[{"f": "verify_password", "to": "pwdOk", "a": [{"n": "password", "t": "str", "v": "..."}]}, {"f": "generate_jwt", "to": "token", "a": [{"n": "userId", "t": "str", "v": "$pwdOk"}]}, {"f": "create_session", "to": "session", "a": [{"n": "token", "t": "str", "v": "$token"}]}
```
6. `parseActionJSON()` → `Action[]`
`validateActionSchema()` → 🍏 结构合法
7. `enrichActions()` → 补充 IR 元数据 (完整类型信息)

8. validateActionSequence():

- └─ SVL-1: verify_password 🍏, generate_jwt 🍏, create_session 🍏
- └─ SVL-2: 参数类型兼容 🍏 (password:str→str, userId:str→str, token:str→str)
- └─ SVL-3: \$pwdOk 已声明 🍏, \$token 已声明 🍏
- 返回 { valid: true, violations: [] }

9. ★ v2.2 validateTransition() 纯函数 SVL-4 验证:

- └─ ctx = { ledger: [], currentState: rebuildState([], nsInit) }
- └─ verify_password: validateTransition(ctx, ...) → auth: UNAUTHENTICATED → PASSWORD_VERIFIED 🍏
- └─ ctx.ledger.push(transition); ctx.currentState = transition.statesAfter
- └─ generate_jwt: validateTransition(ctx, ...) → auth: PASSWORD_VERIFIED → TOKEN_ISSUED 🍏
- └─ ctx.ledger.push(transition); ctx.currentState = transition.statesAfter
- └─ create_session: validateTransition(ctx, ...) → auth: TOKEN_ISSUED → SESSION_ACTIVE 🍏
- └─ 每条转移均写入 ruleHash = "7d8f9a..."
- 协议合法, StateTransition[] 构建完成

10. ★ v2.2 checkLedgerConsistency(ledger) → Invariant-0 + Invariant-1 🍏

11. ★ v2.2 hashLedger(ledger) → 完整性指纹 "b44f80a2b4d1d1f6"

12. checkSemantic() → 🍏 语义合约通过

13. 构建成功 Attempt:

```
{ outcome: "success", violations: [],
  transitions: [StateTransition×3],
  ruleHash: "7d8f9a...",
  antibodyHit: { level: "ACL-3", action: "injected_hint" } }
```

14. recordSession() → 保存 ExecutionSession 到 .progmune_corpus/sessions/
recordEpisode() → 更新记忆层统计

15. emitter.ts 编译为 TypeScript 代码 → 返回给 MCP 客户端

14.2 异常路径示例

如果 LLM 输出的序列未遵循协议（如直接调用 `create_session` 而不先调用 `verify_password` 和 `generate_jwt`），则：

8. `validateActionSequence()` → SVL-1/2/3 通过

9. ★ v2.2 `validateTransition()` SVL-4 验证：

```
|— create_session: auth 命名空间当前状态 = UNAUTHENTICATED
|   create_session 需要 pre_states = [TOKEN_ISSUED]
|   当前状态 ≠ 所需状态 _
|— validateTransition() 返回 { valid: false, transition,
rejection }
|   rejection = {
|       blocked: "create_session",
|       currentState: ["UNAUTHENTICATED"],
|       requiredState: ["TOKEN_ISSUED"],
|       missingFunctions: ["verify_password",
"generate_jwt"],
|       fixPath: ["verify_password", "generate_jwt"] // BFS
搜索结果
|   }
|— transition 仍被记录 (valid=false)，但被标记为无效
```

10. 构建失败 Attempt:

```
{ outcome: "constraint_violation",
  violations: [{ svl: 4, violatedConstraint:
"protocol_violation",
                fixPath: ["verify_password",
"generate_jwt"] }] }
```

11. `attemptSSGRepair()` → 在 `create_session` 前插入 `verify_password`
→ `generate_jwt`

```
→ 合成 Action[]: [verify_password(...), generate_jwt(...),
create_session(...)]
```

→ 重新验证通过 🍏

12. 使用修复后的序列继续 → 成功

13. `recordFailure()` → 记录 SVL-4 违规到失败语料库

(修复成功 + 违规记录 = "感染后免疫"—系统从这次违规中学习)

→ 下次相同意图可直接使用修复路径 (ACL 升级)

14.3 ★ v2.2 Ledger 中心化数据流

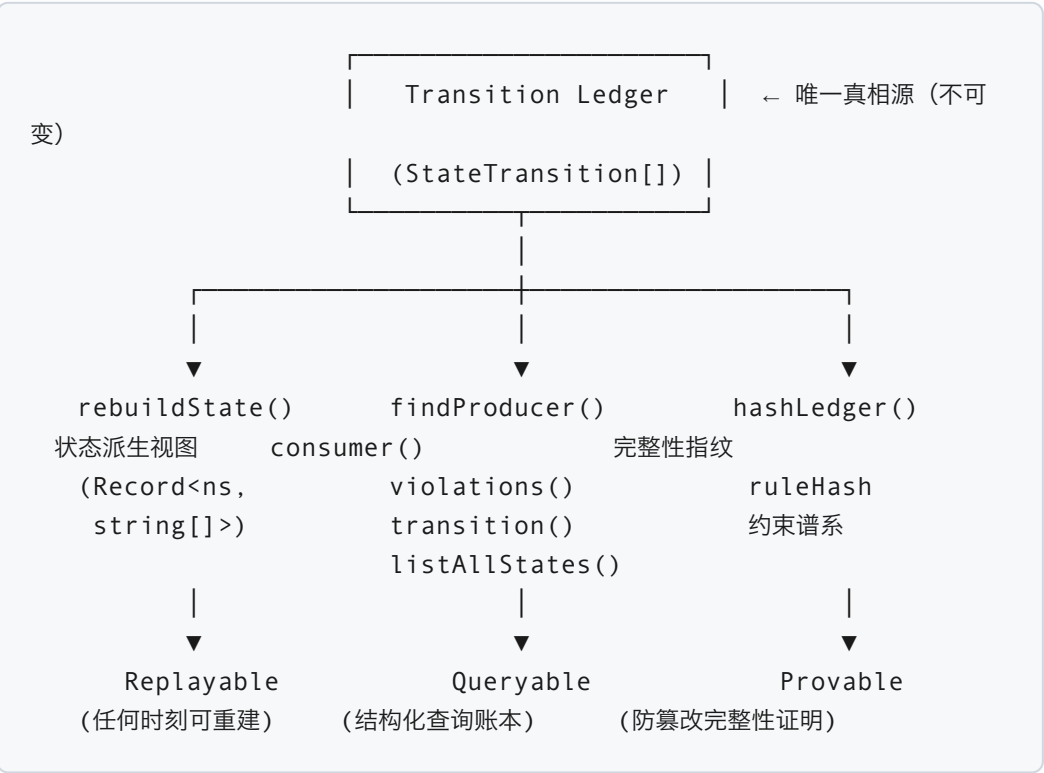


图 14.1: v2.2 以 Ledger 为中心的数据流——状态是派生视图，不是可变对象

附录 A：文件清单 | ★ v2.2 更新

文件	行数 (约)	核心职责	v2.2
src/runtime-types.ts	115	运行时本体论，所有类型的单一来源（含 ValidationContext、LedgerConsistencyViolation）	+ruleHash 在三层接口中， +新类型 re-export
src/ssg-validator.ts	760	SSG 状态机 + Semantic Ledger 16 个纯函数（rebuildState、validateTransition、checkLedgerConsistency、	+~400 行纯函数 + 不变式系统

文件	行数 (约)	核心职责	v2.2
		hashRules、hashLedger、diffLedgers、findProducer、findConsumer 等)	
src/planner.ts	920	核心规划循环，免疫加速，ValidationContext 循环	纯函数化 SVL-4, +ruleHash 注入, +不变式 验证
src/check.ts	380	一键免疫健康检查 (6 步: IR + tsc + SSG + Ledger + Genome + Antibody)	+Ledger 不 变式步骤, +- ledger 单会 话深度验证
src/semantic-trace.ts	1800	终端观测站 (16 个命令: 摘要、时间线、回放、验证、Ledger、查询、聚合查询、统计、差异、基因组、抗体列表、抗体效能、热力图、快照)	+600 行纯函 数查询/统计/ 差异/Ledger 验证
src/p0_ssg_demo.ts	280	SSG 端到端演示 (7 场景: auth 完整流程、dev_pipeline 5 阶段、违规拦截、Invariant 负向测试、Query API、差异比较)	纯函数化, +Ledger 场 景, +Query API 演示
src/validator.ts	205	SVL-1/2/3 约束验证引擎 (validateAction、validateActionSequence、checkVariableFlow)	
src/failure-corpus.ts	650	失败语料库、失败基因组、抗体推理 (LearnedPattern、抗体匹	

文件	行数 (约)	核心职责	v2.2
		配、ACL 等级)、语义热力图、抗体效能统计	
src/extract-ir.ts	407	ts-morph AST 遍历 IR 提取 (函数声明、箭头函数、JSDoc @protocol、调用图、外部函数注册表)	
src/emitter.ts	~110	Action→TypeScript 代码发射 (类型分发: call/assign/return/if/for)	
src/action-runtime.ts	103	Action DSL 执行沙箱 (Proxy 变量管理、自由格式 DSL 解析)	
src/llm.ts	62	LLM 抽象层 (DeepSeek/ OpenAI/Ollama、token 估算)	
src/mcp-server.mts	~200	MCP 工具服务器 (6 个工具: plan / validate / emit / ir_extract / observatory / check)	
src/memory-layer.ts	~80	语义模板记忆层 (模板匹配、episode 记录、成功率统计)	
src/semantic-validator.ts	~60	语义合约校验	
src/semantic-snapshot.ts	~80	IR 快照与差分 (createSnapshot、diffSnapshots、回归检测)	
	~40		

文件	行数 (约)	核心职责	v2.2
src/ feedback.ts		函数成功率统计（用于 LLM prompt 优化）	
src/utils.ts	~30	Jaccard 相似度、关键词提取 （中英文分词、驼峰分词）	
src/file- lock.ts	~30	文件锁（corpus 写入原子性、多 进程互斥）	
src/search- planner.ts	~50	基于搜索的备用规划器（无需 LLM, IR 函数组合搜索）	
src/immune- reporter.ts	~40	免疫报告生成（Failure Genome + Antibody Stats 格式化）	
protocols.json	136	协议定义（17 条规则，4 个命名 空间：_global、auth、file、 dev_pipeline）	

附录 B：配置参考

环境变量	默认值	说明
LLM_PROVIDER	deepseek	LLM 提供商 (deepseek / openai / ollama)
LLM_API_KEY	—	API 密钥
LLM_BASE_URL	提供商默认	自定义 API 端点

环境变量	默认值	说明
LLM_MODEL	提供商默认	模型名称（如 deepseek-chat、gpt-4、llama3）
PROGMUNE_PROJECT_DIR	cwd	项目根目录（多项目隔离的基础）
PROGMUNE_CORPUS_DIR	.progmune_corpus	语料库目录（会话、失败记录、检查点）
PROGMUNE_ACL4_COUNT	10	ACL-4 出现次数阈值
PROGMUNE_ACL4_INTENTS	5	ACL-4 跨意图阈值
PROGMUNE_ACL3_COUNT	4	ACL-3 出现次数阈值
PROGMUNE_ACL3_INTENTS	3	ACL-3 跨意图阈值
PROGMUNE_ACL2_COUNT	2	ACL-2 出现次数阈值

附录 C：术语表 | ★ v2.2 新增

术语	英文	定义	v2.2
IR			

术语	英文	定义	v2.2
	Intermediate Representation	从 TypeScript 源码提取的函数签名中间表示 (FunctionInfo[])，是系统与代码库之间的唯一接口	
SVL	Semantic Validation Layer	四层语义验证层：SVL-1 符号存在性、SVL-2 类型完整性、SVL-3 数据流有效性、SVL-4 协议合法性	
SSG	State-Guided Generation	状态引导生成——将函数调用建模为有限状态机转移的协议验证系统	
ACL	Antibody Confidence Level	抗体置信等级（ACL-1 到 ACL-4），基于出现频次和跨意图数量的自适应分级	
Semantic Ledger	Semantic Ledger	StateTransition[] 作为唯一真相源的事件溯源架构——状态是派生视图，Ledger 是不可变历史	★ 新增
ValidationContext	ValidationContext	{ ledger: StateTransition[], currentState: Record }——O(n) 验证循环的核心数据结构	★ 新增
Invariant-0	Before Consistency	前置一致性不变式： transition.statesBefore ≡ rebuildState(ledger[0..i-1]), 保证转移与历史一致	★ 新增
Invariant-1	Delta Consistency	增量一致性不变式： transition.statesAfter ≡ applyDelta(statesBefore,	★ 新增

术语	英文	定义	v2.2
		acquired, invalidated), 保证转移内部自治	
ruleHash	Constraint Snapshot	协议规则集的 SHA256 哈希, 存储在 StateTransition、Attempt、ExecutionSession 三个层级, 实现跨版本确定性回放验证	★ 新增
hashLedger	Ledger Fingerprint	整个 StateTransition[] 账本的防篡改 SHA256 完整性指纹, 在 check.ts 中验证	★ 新增
diffLedgers	Ledger Diff	两个账本的结构化差异比较 (不变/仅A/仅B/变更转移数), 用于跨会话审计	★ 新增
Event Sourcing	Event Sourcing	以不可变事件序列为真相源的架构模式——v2.2 的核心架构跨越, 从 Event Logging 进化为 Event Sourcing	★ 新增
Action	—	程序行为的原子单位 (call/assign/return/if/for), LLM 输出的最小单元	
Attempt	—	规划器的单次推理尝试 (LLM 调用 + 验证 + 可能的修复), 一次会话可包含多次尝试	
StateTransition	—	协议状态机的单次合法 (或非法) 转移, 携带完整的前后状态快照和增量变化	
ConstraintViolation	—		

术语	英文	定义	v2.2
		约束违规的结构化记录（SVL 层级、违规类型、修复路径、命名空间等诊断信息）	
ExecutionSession	—	一次完整的规划会话，包含所有 Attempt、最终结果和时间线	
AntibodyHit	—	抗体命中记录（ACL 等级、签名、修复路径、相似度、节省的 LLM 调用和 token 数）	
Failure Genome	—	所有失败记录的统计画像（按 SVL/约束类型/修复路径的分布），终端可通过 --genome 查看	
Semantic Heatmap	—	语义脆弱点的四维热度分析（脆弱协议、SVL 热点、约束集群、高摩擦意图）	
Cognitive Replay	—	规划器尝试序列的逐步动画回放，展示每次尝试的动作、违规、修复和最终成功路径	
Strangler Pattern	—	v2.2 使用的向后兼容策略：保留 StateMachineValidator 类但内部委托给纯函数，新代码直接使用纯函数	★ 新增